



CONTENTS

1	Trees	3
1.1	Binary Trees and Binary Search Trees	4
1.2	File Structures and File System Hierarchies	6
1.3	Red-Black Trees and AVL Trees	7
1.4	B-Trees, Tries, and Suffix Trees	8
2	Tree Traversal	9
2.1	BFS	9
2.1.1	Pseudocode	11
2.2	DFS	12
2.2.1	Pre-order	12
2.2.2	Post-order	13
2.2.3	In-order	13
2.2.4	Pseudocode	14
2.3	Recursive vs Iterative Traversal	14
2.3.1	Pre-order and Post-order Numbering	15
2.4	Proofs	16
3	Finding Shortest Paths	17
3.1	Introduction — BFS revisited	17
3.2	Dijkstra's Algorithm	18
3.2.1	Algorithm Description	18
3.2.2	Implementation	21
3.3	Bellman-Ford Algorithm	24

4	Priority Queues	27
5	Other Graph Algorithms and Proofs	29
5.1	Adjacency Matrices	29
5.2	Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees	32
5.3	A* (A-star) Search Algorithm	32
5.3.1	The Path Cost $g(n)$	32
5.3.2	The Heuristic Function $h(n)$	33
5.3.3	The Total Estimated Distance $f(n)$ and Priority Queues	34
5.4	Injection, Surjection, Bijection, and Isomorphism	34
5.5	Travelling Salesperson Problem and Nearest Neighbour	38
5.6	Topological Sort and DAGs	38
5.7	Proofs	41
5.7.1	Example 1: Bipartite	41
5.7.2	Example 2 and 3. Cycle Detection and Leaves	42
A	Answers to Exercises	45
	Index	51

Trees

Trees are one of the most versatile and widely used data structures in computer science. A *tree* is a hierarchical data structure consisting of nodes, where each node has a value and a set of references (*edges*) to its child nodes. The node at the top of the hierarchy is called the *root*, and nodes with the same parent are called *siblings*, as seen in Figure 1.1.

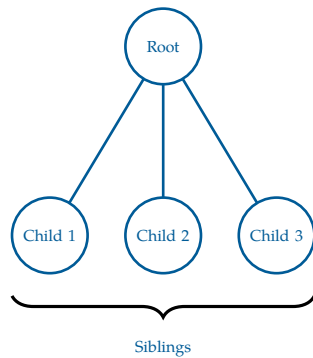


Figure 1.1: A tree with a root node and 3 children.

Trees are a subset of Graphs, a data structure comprised of nodes and edges. Graphs can be converted into trees by adding the following two constraints:

- The graph is connected, such that there exists a path between every pair of nodes
- The graph is acyclic; no cycles exist in the graph

With this comes the caveat that the number of edges in a tree is $\text{edges} = n - 1$, where n is the number of nodes in the tree. This is because a tree with n nodes must have exactly $n - 1$ edges to maintain its connected and acyclic properties. If there were more than $n - 1$ edges, it would create a cycle, violating the acyclic property. If there were fewer than $n - 1$ edges, it would not be fully connected, violating the connected property.

Less formally, a *forest* is a collection of multiple trees (may be disconnected), and a *tree* is a forest with only one tree. Lots of agricultural terms in this chapter!

Trees have a unique structural property: between any two nodes, there exists exactly one simple path. This follows directly from the fact that trees are both connected and acyclic. As a result, there is only one way to move from a start node to an end node.

Although trees are often drawn with a root at the top, this is a matter of convention. Any node in a tree can be chosen as the root, and doing so induces a hierarchical structure in which edges are interpreted as parent-child relationships.

Once a root is selected, every node (except the root) has exactly one parent, and zero or more children. Because of this relationship, any randomly selected node can form a tree (we call this a *subtree*, consisting of the node and all its descendants), making the tree structure *recursive*. Moving from one node to the next is the recursive step that minimizes

the size.

Nodes with no children are called, as you'd expect, *leaves*, as they are the outermost node of a tree. The *depth* of a chosen node is the number of edges from the *root* to that node, and the *height* of a tree is the maximum depth among all nodes. Equivalent to the height of a tree is the maximum number of edges from the *root* to any leaf. You may see the *level* of a node referred to as the number of edges from the *root* to that node, similar to depth. By convention, the level and depth of the root is 0. See Figure 1.2.

Some more relevant terminology you may encounter in the context of trees:

- *Path*: A sequence of nodes where each adjacent pair is connected by an edge. This can be used to show the idea of how to get between two nodes in a tree, for example, between two cities in a flight path network.
- *Ancestor*: A node that lies on the path from the root to a given node. This is done by repeatedly traversing from a child node to a parent node until the selected node is reached.
- *Descendant*: A node that lies on the path from a given node to a leaf. This is done by repeatedly traversing from a parent node to a child node until a leaf is reached.

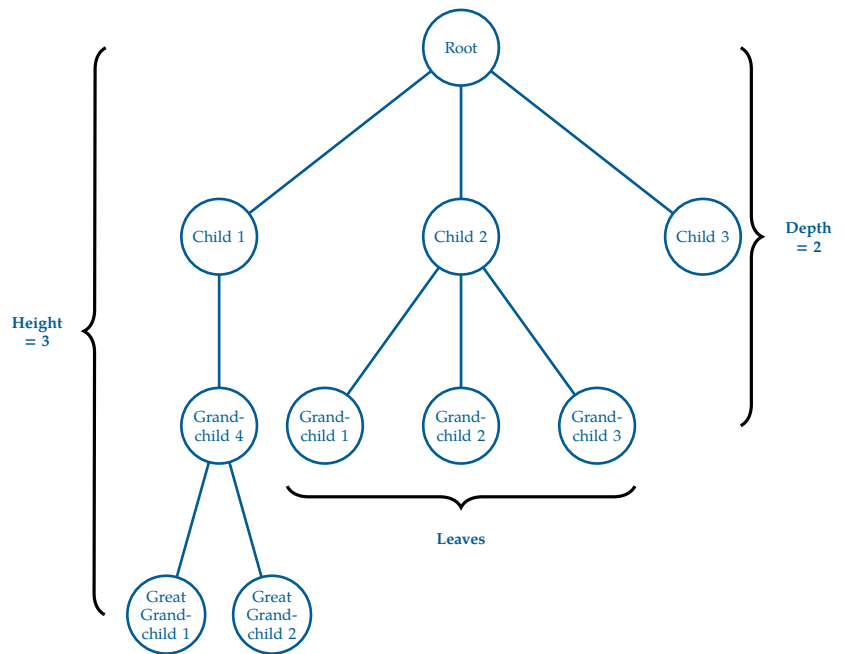


Figure 1.2: A tree with a root node and 3 children, a few grandchildren, and 2 great grandchildren.

FIXME tree proof

1.1 Binary Trees and Binary Search Trees

The most common type of tree is the *binary tree*, where each node has *at most* two children, referred to as the *left child* and the *right child*. Binary trees are widely used in various applications, including expression parsing, binary search trees, and heaps.

Let's put this practice. Recall our binary search implementation from the recursion chapter. Given a chosen word and a sorted list of words, such as a dictionary, we can find the page of the chosen word by repeatedly dividing the available pages in half and comparing

the elements on the page to the chosen word. This is much easier than, for example, going through every single page and checking if the word is on that page.

Say we wanted to represent our dictionary as a tree that we could recursively binary search through. For simplicity, we will limit our words to mango, apple, pear, banana, anagram, peony and peach. These 7 words form a binary tree of height 2, as seen in Figure 1.3. We select the word mango as the root as it is the middle word after sorting alphabetically.

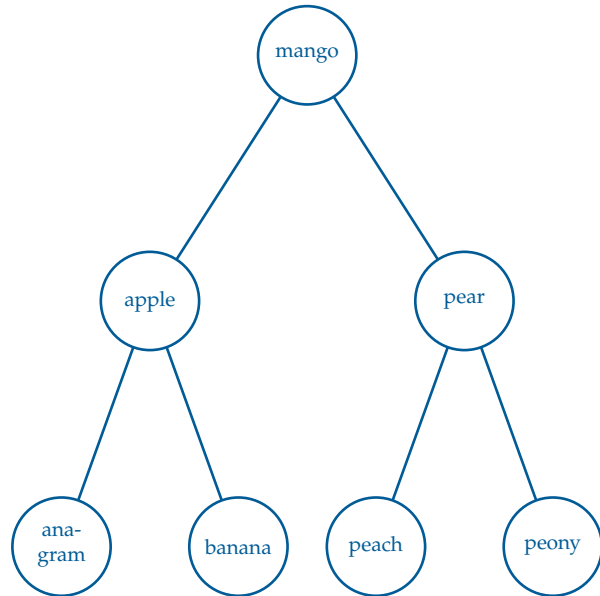


Figure 1.3: A binary search tree with 7 words.

Similar to binary search, we can find any word in the tree by starting at the root and comparing the target word to the current node's word. If the target word is less than the current node's word, we move to the left child; if it is greater, we move to the right child. We repeat this process until we find the target word or reach a leaf node, which tells us that the target word is not in the tree. This structure allows us to cut the available words in half with each step in the tree we go.

What if we want to insert into a binary search tree? We must complete four actions:

- Find the correct location for the new node.
- Create and insert the node at that location,
- Connect it to its respective parent(s).
- Ensure the BST properties are maintained.

Let's say, on our tree above, we want to add orange to the tree. Because of alphabetical sorting,

FIXME expand on addition, removal, and search of binary trees
FIXME add exercise of a similar dictionary tree with more words, have student add and remove words, and search for words

1.2 File Structures and File System Hierarchies

Computers, since their first creation in the 1980s, have needed a way to store and organize data. Prior to computers, many systems of organization were used to store physical data, such as filing cabinets, libraries, and warehouses.

These systems often had a hierarchical structure, with a top-level category that branched out into subcategories, with these subcategories further branching out into more specific categories, and so on.

For example, a library might have two top-level categories: fiction and non-fiction. Within the category non-fiction, you may find psychology, biographies, autobiographies, self-help, journalism books, or even nature guides. Under the category fiction, you may find romance, classics, crime, science fiction, thriller, or fantasy novels. Of course, these sub-genres can have even further categorizations too.

A hospital may have patient records categorized by first letter of last name, and then sorted alphabetically from there.

With the development of the computer, sorting these types of hierarchical data became easy with the concept of directories. A *directory*, also called a *folder*, is a categorical way of organizing data (for example, by storing files or other subdirectories). We will stick to the more technical term of directories, however, any time you create a folder on your computer, you are making a *directory*. The name comes from how, in the twentieth-century, many phone books were referred to as telephone directories.

This structure naturally fits into the tree data structure:

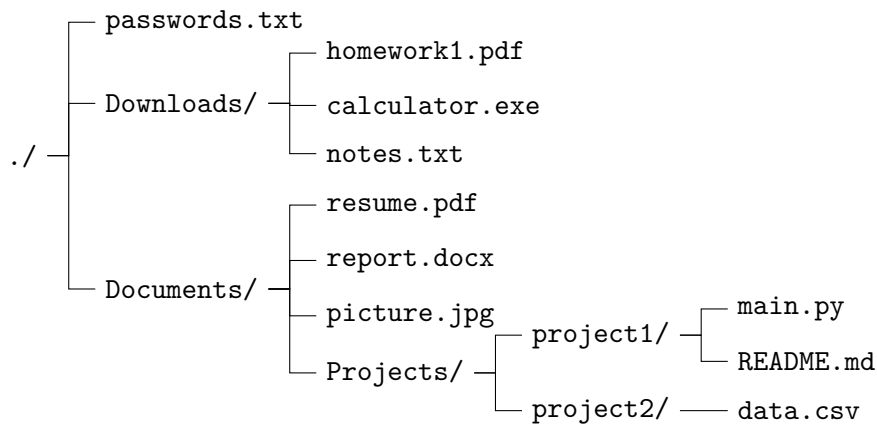
- The **root** of the tree is the top-level directory (/ on Linux/Unix¹ systems or C:\ on Windows).
- A **node** is naturally fits the idea of either *directories nodes* or *files nodes*.
- A **directory node** may have children, where its children are subdirectories or files.
- A **file node** is a **leaf**, meaning it has no children.

The **path** to a file describes the unique sequence of directories from the root to that file.
Test again test again test again

Example

A small file system might look like:

¹There is a very helpful video in Linux File Systems in your digital resources. Check it out!



In tree terminology:

- `./` is the root node.
- Downloads, Documents, and its subdirectories are internal nodes (directories).
- All files, such as those ending in `.txt`, `.png`, `.exe`, `.csv`, `.docx`, `.py` files are leaf nodes.

Side note: please do not store your passwords in anywhere on your computer's memory!

Exercise 1 File Paths

Working Space

In the above File Hierarchy Tree, what is the path to `data.csv`? What is the depth of the file tree at that point?

Answer on Page 45

1.3 Red-Black Trees and AVL Trees

1.4 B-Trees, Tries, and Suffix Trees

Tree Traversal

Trees can be “traversed” in a few different ways. When we say traversing a tree, we mean that we are visiting and extracting data from each node in a tree. We classify these *traversals* based on the order in which the nodes are visited.

This chapter will discuss *breadth-first search*, also known as *BFS*, which traverses in a *level-order*. Each level is traversed in sequential order.

Alternatively, there is *depth-first search* (*DFS*), which travels as far down a branch as possible before backtracking and checking other branches. Within DFS there are 3 common traversal methodologies: *in-order*, *pre-order* and *post-order*.

While the demonstrations for these traversals will be for trees, they work the same for any given graph, under one condition: using DFS or BFS on a *graph* requires you do have a preselected node a your *start node*.

2.1 BFS

Let’s look at running BFS on a tree. Starting out with this tree:

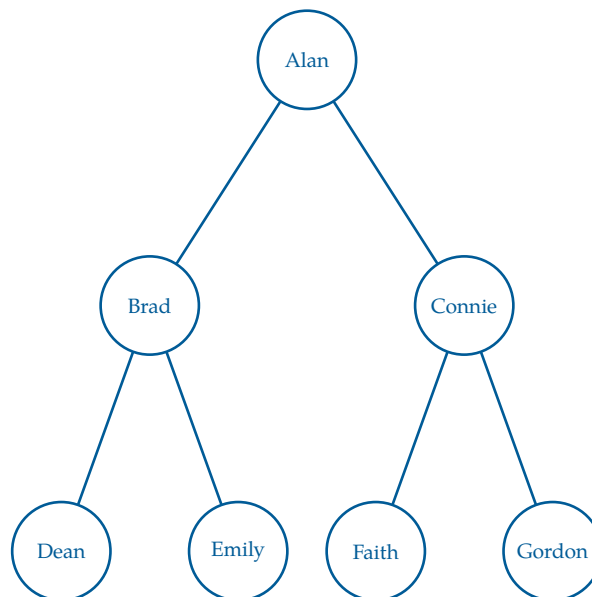


Figure 2.1: Our original tree containing a list of names to be traversed.

To perform BFS, we use a *queue*, which follows a first-in, first-out (*FIFO*) order. We begin by placing the root node into the queue. At each step, we remove the front node from the queue, visit it, and then add all of its children to the *back* of the queue.

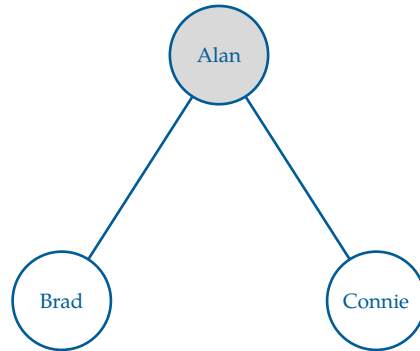


Figure 2.2: Step 1: Visit Alan, then add its children (Brad, Connie) to the queue.

Here, we visit the root node first, which is Alan. After visiting Alan, we add its children, Brad and Connie, to the queue. Since there are no other nodes to visit at this level, we move on to the next level of the tree.

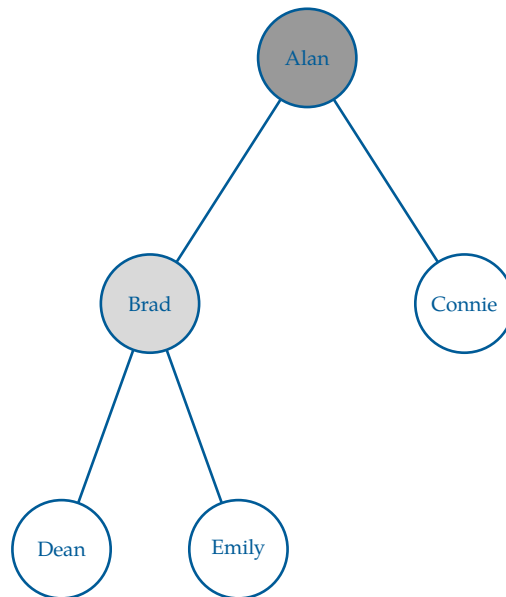


Figure 2.3: Step 2: Visit Brad, then add its children (Dean, Emily) to the queue.

We continue this process, visiting Connie next and adding its children, Faith and Gordon, to the queue. The queue now contains Connie, Dean, and Emily. Here is a step trace of the BFS traversal:

Step	Queue	Output (visited)
Start	[Alan]	[]
1	[Brad, Connie]	[Alan]
2	[Connie, Dean, Emily]	[Alan, Brad]
3	[Dean, Emily, Faith, Gordon]	[Alan, Brad, Connie]
4	[Emily, Faith, Gordon]	[Alan, Brad, Connie, Dean]

Continuing this process until the queue is empty gives the final traversal:

Alan, Brad, Connie, Dean, Emily, Faith, Gordon

This ordering reflects a level-by-level traversal of the tree, which is why BFS on a tree is also called level-order traversal.

To use BFS for *searching* rather than full traversal, we simply introduce a target value and stop the algorithm as soon as the target is found. Instead of visiting every node, we check each node when it is dequeued and terminate early if it matches the desired value.

For example, suppose we want to find “Faith” in the tree. BFS will examine nodes level by level: Alan, then Brad and Connie, then Dean, Emily, and Faith. As soon as Faith is encountered (we *dequeue* it from the queue), the search stops, without visiting any remaining nodes.

2.1.1 Pseudocode

Our steps can be reflected as pseudocode as follows:

Algorithm 1 Breadth-First Search (BFS) - Iterative

```

procedure BFS(root, target)
  Q = empty queue
  ENQUEUE(Q, root)
  while Q is not empty do
    node = DEQUEUE(Q)
    if node = target then
      return node
    end if
    for each child of node do
      ENQUEUE(Q, child)
    end for
  end while
  return null
end procedure

```

This procedure is often used in game applications to find paths, such as maze solvers, chess algorithms, and tic-tac-toe solutions. It is also used in social network analysis to find the shortest path between two users, and in web crawling to explore the structure of websites.

2.2 DFS

In contrast, we can run a *depth-first search* on the original tree (see Figure 2.1). Depth-first search, as the name implies, goes as deep down the tree until reaching a leaf, and then backtracks to the next possible, unsearched path. DFS (implicitly) uses a stack to explore the graph or tree.

The steps are fairly simple:

1. Push the root node into the stack.
2. Pop a node from the stack, and mark it as visited.
3. Push the node's children onto the stack.
4. Repeat steps 2 and 3 until the stack is empty.

There are 3 orders in which we can traverse/search the deepest tree. We will look at pre-order, post-order, and in-order.

However, these orders essentially perform the same basic three tasks,

V Visit the selected node.

L Run DFS on the selected node's left subtree.

R Run DFS on the selected node's right subtree.

The recursion terminates when we reach a leaf. At this point, the algorithm returns to the previous call, which is how *backtracking* occurs. This backtracking is done via the stack we had mentioned previously.

2.2.1 Pre-order

In pre-order traversal, we visit the node *before* exploring its subtrees. The order is:

(V, L, R)

We visit the node *before* its children because the node's value is needed *prior* to exploring its subtrees. This is useful when the node represents a decision that affects which branch of the tree you go down, such as in tree copying or directory traversal like we talked about in the previous chapter.

Using our original tree, the pre-order results are:

Alan, Brad, Dean, Emily, Connie, Faith, Gordon

2.2.2 Post-order

In post-order traversal, we visit the node *after* exploring both subtrees. The order is:

(L, R, V)

We visit the node *after* its children because the node's computation depends on results from both subtrees. This is useful when combining subtree results or deleting trees, where children must be handled before the parent.

The post-order traversal of our original tree is

Dean, Emily, Brad, Faith, Gordon, Connie, Alan

2.2.3 In-order

In in-order traversal, we visit the node *between* exploring its subtrees. The order is:

(L, V, R)

We visit the node *between* its subtrees because the correct interpretation of the node depends on partially processed children. In-order only makes sense for binary trees, because it returns the *sorted order* of the binary search tree. It depends on left and right subchildren. This would be like the alphabetical order of our dictionary example from the previous chapter.

The in-order traversal of our original tree becomes

Dean, Brad, Emily, Alan, Faith, Connie, Gordon

2.2.4 Pseudocode

The previous sections (pre-order, in-order, and post-order) describe *when* a node is visited during a depth-first traversal.

The following algorithm instead describes the general *mechanism* of depth-first search. This implementation corresponds to a pre-order traversal. Other traversal orders (in-order and post-order) require additional state and cannot be obtained by simple reordering alone.

Algorithm 2 Depth-First Search (DFS) - Iterative on a Tree

```
procedure DFS(root)
  S = empty stack
  PUSH(S, root)
  while S is not empty do
    node = POP(S)
    visit(node)                                ▷ V
    if node.left ≠ null then
      PUSH(S, node.left)                       ▷ L
    end if
    if node.right ≠ null then
      PUSH(S, node.right)                      ▷ R
    end if
  end while
end procedure
```

DFS is particularly useful for solving problems like connected-component detection in graphs and maze-solving.

2.3 Recursive vs Iterative Traversal

As we've established, trees are recursively defined structures, and as such, our algorithms for them can be defined as such too. This allows us to implement searches and traversals in two ways: *recursive* and *iterative*. Let's look at DFS recursively.

Importantly, they do not change the traversal order (pre-, in-, post-order); they only change how the traversal is implemented.

A *recursive traversal* relies on the program's *call stack* (recall from the Advanced Python Syntax chapter). Each time the algorithm explores a subtree, it makes a function call, and the system automatically remembers where to return when that call finishes. This matches the recursive definition of a tree, where each subtree is itself a tree. This allows us to get rid of the stack in DFS and solely rely on a computer's call stack.

Exercise 2 Recursive DFS

Take a look at the Iterative DFS code above.

Working Space

How can we rewrite the pseudocode to be recursively define, taking advantage of a inherent recursive structure? Try pre-order recursively first. Assume the tree is a binary tree.

Answer on Page 45

Note that we can create recursive DFS just by reordering the three operations (V, L, R).

What about BFS? BFS is typically implemented iteratively because it relies on a queue. While recursive versions exist, they still depend on a queue and offer little conceptual advantage, so we omit them here.

2.3.1 Pre-order and Post-order Numbering

There also exists pre-order and post-order numbering, regardless of which version of DFS is run. Pre-order and post-order numbering are techniques used during a depth-first search (DFS) traversal to record when each node is encountered and when it is fully processed. A *pre-order number* is assigned when the traversal first visits a node, before exploring any of its children. A *post-order number* is assigned after all of the node's descendants have been explored and the traversal is about to return to the node's parent. These numbers can be viewed as *timestamps* that describe the progress of the DFS.

Pre-order numbers capture the order in which nodes are discovered, while post-order numbers capture the order in which nodes are completed. This information is useful in many graph and tree algorithms, including dependency analysis, topological sorting (which we will talk about in the future), and determining ancestor-descendant relationships.

Exercise 3 DFS Pseudocode

Alter the *recursive* DFS pseudocode to include an assignment of preorder and post order numbers for every node.

Working Space

Answer on Page 46

Exercise 4 Pre- and Post-order numbers

Write the pre and post order numbers for the person graph we have been working with, Figure 2.1

Working Space

Answer on Page 46

2.4 Proofs

Finding Shortest Paths

3.1 Introduction — BFS revisited

We have seen BFS as a way to search graphs (or trees). It is a pretty simple process:

- select a start node
- move one distance from the root, and visit all nodes (v) on that level
- when visiting, queue all child nodes (v') of that node.
- keep going until either all nodes are marked *visited* or a provided *target node* is reached.

There is an issue here: what if the graph is weighted, and the paths are not equally weighted? For example, see this graph:

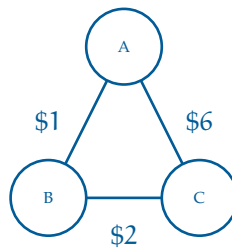


Figure 3.1: A graph of 3 nodes that would fail BFS.

BFS, here, would prefer $A \rightarrow C$, since it has an edge count of 1, whereas $A \rightarrow B \rightarrow C$ has an edge count of 2. However, the true shortest path, by weight, is $A \rightarrow B \rightarrow C$, with a weight of 3.

So BFS has a major limitation: it cannot handle weighted graphs. This is why we started it out on trees; typically trees do not have a set edge weight, so we were not limited by the order in which it expands.

To handle weighted graphs, we use a new algorithm that expands off of BFS, called *Dijkstra's Algorithm*. Dijkstra's Algorithm avoids the limitation of weights that BFS had by enqueueing the nodes in order of current shortest known distance.

3.2 Dijkstra's Algorithm

Edsger W. Dijkstra (pronounced *DYKE-strah*) was a great Dutch computer scientist. He came up with an algorithm for finding the cheapest path through a graph with weighted edges. Today it is known as *Dijkstra's Algorithm*. It is used in a wide variety of common problems, and is also both simple and elegant.

3.2.1 Algorithm Description

You are going to mark each node with how much it would cost to get there from some chosen origin node. For example, if you are shipping a container from Long Beach, you will mark each city with the cost of getting the container to that city.

You start by marking the price for Long Beach to zero (the container is already there). You then mark each adjacent city with the cost on the edge (figure 3.2a). Next, you declare Long Beach to be “visited” (figure 3.2b).

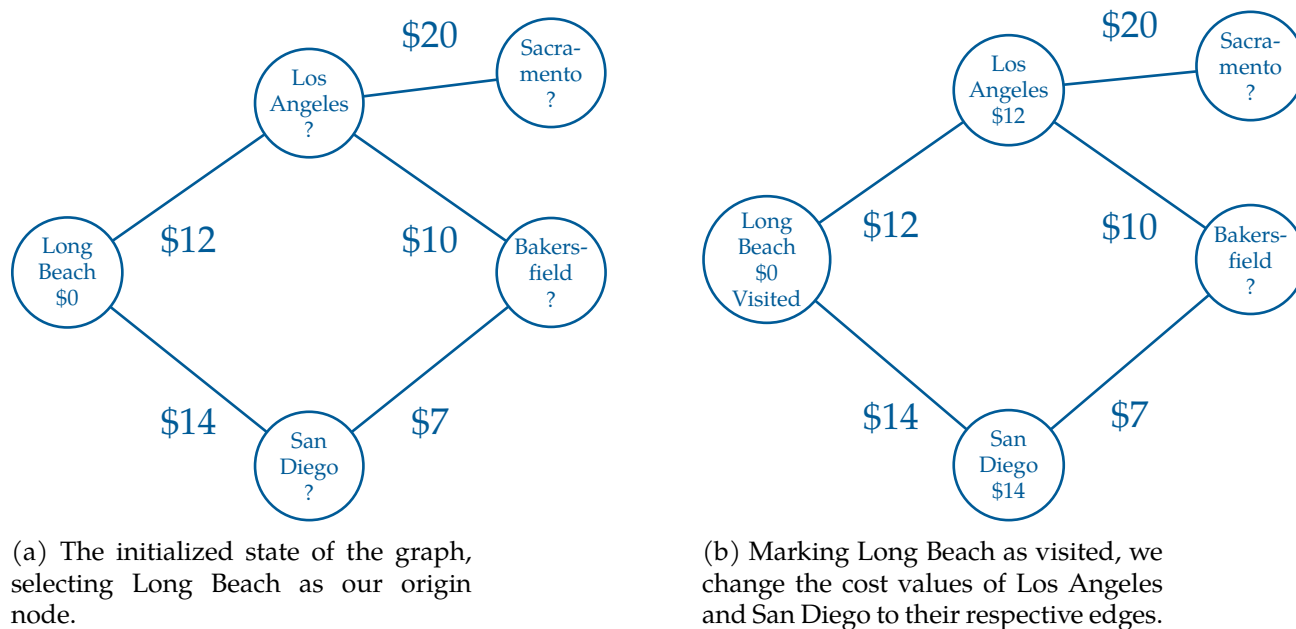


Figure 3.2: A map of weighted cities.

After that, you find the cheapest of the unvisited nodes. In this case, Los Angeles is cheaper than San Diego, so that is the node you will visit next.

You mark all of the unvisited nodes adjacent to Los Angeles, with the price to ship it to Los Angeles plus the cost of shipping the container from Los Angeles to that city (figure

3.3a). Note that Bakersfield is marked with \$22.

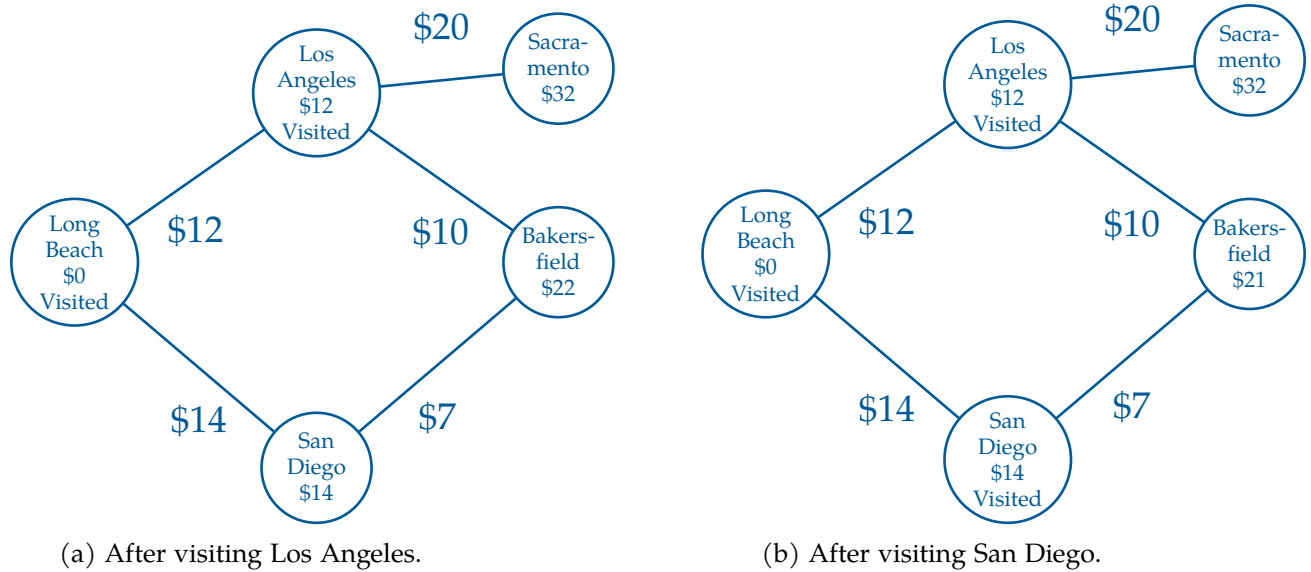


Figure 3.3: Visiting the next two minimum cost cities.

Now, the cheapest unvisited node is San Diego, so you mark its neighbors with the cost to ship to San Diego, plus the price to ship from San Diego to the neighbor (figure 3.3b).

Notice that Bakersfield is already labeled with \$22 from a route through Los Angeles. But the price would be \$21 if you shipped it to Bakersfield via San Diego. Because the new route is cheaper, you change the price to the lower value.

(What does it mean that a node is “visited”? If a node is marked visited, it is marked with a price that won’t get any smaller.)

You continue visiting the cheapest unvisited node until all the nodes have been visited. Then you know every node has been marked with its lowest price.

In a big graph, each node may be marked several times in this process — each time with a lower price from a cheaper router.

Of course, once you have the price, you will ask, “What is the route that gets me that price?” So, we will also mark each node with the neighbor from which it would receive the shipment — the previous node. This is easy to do as we execute the algorithm.

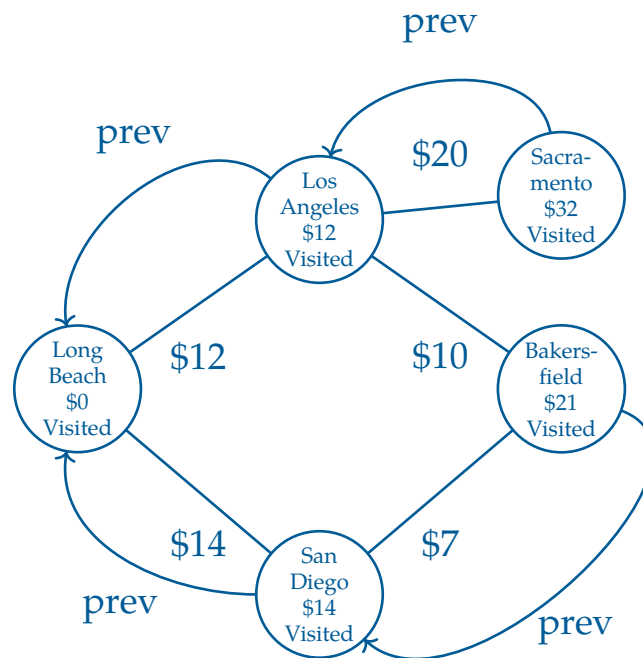


Figure 3.4: Marking each node with a previous pointer.

Now, to figure out the cheapest route from Long Beach to Bakersfield, we start at the destination and follow the prev pointer back through San Diego, then to Long Beach.

Exercise 5 Expanding on the Delivery Graph

Notice that the path between Los Angeles to Bakersfield is not used once we add in our previous labels. Why might this be?

Working Space

Answer on Page 46

3.2.2 Implementation

We do not actually want to sully our graph objects with the three additional pieces of information we need:

- The current minimal cost from the origin node. This is usually called the `dist`, from “distance”.
- The neighbor who gives us the current minimal cost. This is usually called `prev`, from “previous”.
- Whether the city is visited or now.

So, we will keep them in collections external to the graph.

For example, to keep track of the `dist`, we will have a `dist` dictionary. Each node will be a key, the current minimal cost will be the value. If the node hasn't received even a first cost, we will put in infinity as the cost.

(After the algorithm is run, if the cost of a node is still infinity, that means that it cannot be reached from the origin node.)

We will also have a `prev` dictionary. The final node will be the key, and its previous neighbor will be the value.

Finally, the graph has a list of all the nodes, so we can just keep a set of the unvisited nodes.

Add a method to the `Graph` class that implements Dijkstra's algorithm:

```

1  def cost_from_node(self, origin_node):
2      # Cost of cheapest path from origin node discovered so far
3      # Initially the origin is zero and all the other are infinity
4      dist = {k: math.inf for k in self.nodes}
5      dist[origin_node] = 0.0
6
7      # The previous city on that cheapest path
8      prev = {}
9
10     # All the nodes start as unvisited
11     unvisited = set(self.nodes)
12
13     # While there are still unvisited nodes
14     while unvisited:
15
16         # Find unvisited node with lowest cost
17         min_cost = math.inf

```

```

18     for u in unvisited:
19         if dist[u] < min_cost:
20             current_node = u
21             min_cost = dist[u]
22
23     # If none are less than inf, we are done
24     # This happens in graphs that are not connected
25     if min_cost == math.inf:
26         return (dist, prev)
27
28     # Remove the lowest cost node from the unvisited list
29     unvisited.remove(current_node)
30
31     # Update all the unvisited neighbors
32     for edge in current_node.edges:
33
34         # What node is at the other end of this edge?
35         v = edge.other_end(current_node)
36
37         # Visited nodes are already minimized, skip them
38         if v not in unvisited:
39             continue
40
41         # Is this a shorter route?
42         alt = dist[current_node] + edge.cost
43         if alt < dist[v]:
44
45             # Update the distance and prev dicts
46             dist[v] = alt
47             prev[v] = current_node
48
49     return (dist, prev)

```

Each time we visit a node, we have a decision to make:

if $\text{dist}[u] + w(u, v) < \text{dist}[v]$ then update $\text{dist}[v]$

This action, the core foundation of Dijkstra's, is done in lines 16-21. In Dijkstra's, the node with the minimum tentative distance can also be finalized, provided all edge weights are non-negative. Nodes left at infinity are not reachable from the chosen root node.

Append some code to your `cities.py` that tests this method:

```

1  (cost_from_long_beach, prev) = network.cost_from_node(long_beach)
2  print(f"\nMinimum costs from Long Beach = {cost_from_long_beach}")
3  print(f"\nLast city before = {prev}")
4
5  nyc_cost = cost_from_long_beach[nyc]
6
7  if nyc_cost < math.inf:

```

```

8     print(f"\n*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
9 else:
10    print("You can't get to NYC from Long Beach")

```

When you run it, you should get a list of how much it costs to ship a container to each city from Long Beach:

```

1 Minimum costs from Long Beach = {(node:Long Beach, edges:1): 0.0,
2 (node:Los Angeles, edges:3): 12.0, (node:Denver, edges:3): 35.0,
3 (node:Phoenix, edges:3): 31.0, (node:Louisville, edges:3): 49.0,
4 (node:Cleveland, edges:4): 44.0, (node:Boston, edges:2): 57.0,
5 (node:New York City, edges:3): 52.0}

```

You will also get a collection of node pairs. What are these? For each node, you get the node that you would pass through on the cheapest route from Long Beach:

```

1 Last city before = {(node:Los Angeles, edges:3):(node:Long Beach, edges:1),
2 (node:Denver, edges:3):(node:Los Angeles, edges:3),
3 (node:Phoenix, edges:3):(node:Los Angeles, edges:3),
4 (node:Louisville, edges:3):(node:Denver, edges:3),
5 (node:Cleveland, edges:4):(node:Denver, edges:3),
6 (node:New York City, edges:3):(node:Cleveland, edges:4),
7 (node:Boston, edges:2):(node:Cleveland, edges:4)}

```

Your users will not want to read this; give them the shortest path as a list. Add a function to `graph.py` that turns the `prev` table into a path of nodes that lead from the origin to the destination:

```

1 def shortest_path(prev, destination):
2
3     # Include the destination in the path
4     path = [destination]
5     current_node = destination
6
7     # Keep stepping backward in the path
8     while current_node in prev:
9
10        # What node should come before the current node?
11        previous_node = prev[current_node]
12
13        # Insert it at the start of the list
14        path.insert(0, previous_node)
15        current_node = previous_node
16
17    return path

```

Test that out:

```

1 if nyc_cost < math.inf:
2     print(f"*** Total cost from Long Beach to NYC: ${nyc_cost:.2f} ***")
3
4     path_to_nyc = graph.shortest_path(prev, nyc)
5     print(f"*** Cheapest path from Long Beach to NYC: {path_to_nyc} ***")
6 else:
7     print("You can't get to NYC from Long Beach")

```

This should look like this:

```

1 *** Cheapest path from Long Beach to NYC: [(node:Long Beach, edges:1),
2 (node:Los Angeles, edges:3), (node:Denver, edges:3), (node:Cleveland, edges:4),
3 (node:New York City, edges:3)] ***
4 \end{text}
5
6 % \subsection{Making it faster}
7
8 % On really big networks, doing a full Dijkstra's algorithm would take
9 % too long; luckily, there are many methods for getting similar results
10 % quickly. When you ask for directions from Google Maps, it does not do
11 % a full Dijkstra's Algorithm for every possible route --- it would just
12 % take too long.
13
14 % However, there is a way to speed up this implementation. Look at this snippet:
15
16 % \begin{minted}{python}
17 %     # Find unvisited node with lowest cost
18 %     min_cost = math.inf
19 %     for u in unvisited:
20 %         if dist[u] < min_cost:
21 %             current_node = u
22 %             min_cost = dist[u]

```

3.3 Bellman-Ford Algorithm

The Bellman-Ford Algorithm is another shortest path algorithm like Dijkstra's. However, unlike Dijkstra's Algorithm, Bellman-Ford can handle graphs with negative weight edges.

The algorithm works as follows:

1. Assign a tentative distance value for every node: set it to zero for our initial node and to infinity for all other nodes.
2. For each edge (u, v) with weight w , if the current distance to v is greater than the

distance to u plus w , update the distance to v to be the distance to u plus w .

3. Repeat the previous step $|V| - 1$ times, where $|V|$ is the number of vertices in the graph.
4. After the above steps, if you can still find a shorter path, there exists a negative cycle.

If the graph does not contain a negative cycle reachable from the source, the shortest paths are well-defined, and Bellman-Ford will correctly calculate them. If a negative cycle is reachable, no solution exists, but Bellman-Ford will detect it.

Priority Queues

Other Graph Algorithms and Proofs

This chapter is an overview of other graph algorithms, proofs, and concepts that are important to know. We will first cover adjacency matrices, then Prim's algorithm for minimum spanning trees, isomorphism, and topological sorting for directed acyclic graphs (DAGs). A lot of these are very advanced graph theory topics, so you won't be tested on too heavily of these topics. Many of these appear on AP/IB exams, or in advanced college level CS classes.

5.1 Adjacency Matrices

Recall that we can represent graphs as lists of data called adjacency lists. In code, this could look like the following:

- A: [B, C]
- B: [A, D]
- C: [A]
- D: [B]

This represents the following graph

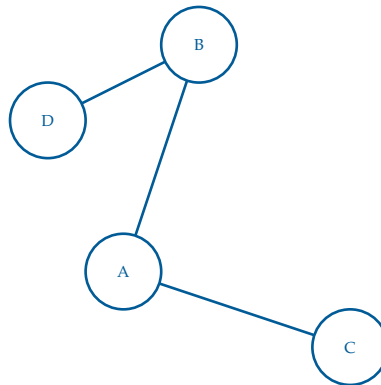


Figure 5.1: A simple (undirected unweighted) graph used to make an adjacency list and adjacency matrix.

We could also represent the same data as a *matrix*, which we have covered in linear algebra. The *adjacency matrix* is 2D array (as compared to an adjacency list, which consists of multiple 1D arrays) where $\text{matrix}[i][j]$ indicates whether an edge exists from vertex i to vertex j . Commonly, $\text{matrix}[i][j] = 1$ if and only if the edge from vertex i to vertex j exists, else $\text{matrix}[i][j] = 0$.¹ The above graph in Figure 5.1 can be represented as:

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

For *undirected* graphs, the adjacency matrix is symmetric, and thusly $n \times n$, meaning that the transpose of matrix is the same matrix!

$$A = A^T$$

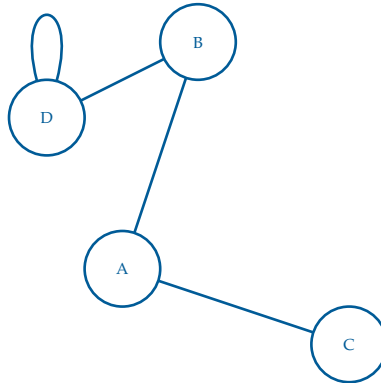


Figure 5.2: A simple graph with an added loop on vertex D.

By convention, a diagonal entry such as $\text{matrix}[k][k]$, where $(k \in \mathbb{Z})$, is non-zero only if there is a loop from a vertex to itself. The value of the entry represents the number of such loops. In this case, there are two edges from vertex (D) to itself in Figure 5.2, so the corresponding diagonal entry is (2).

For *directed* graphs, the adjacency matrix on appears a bit different. A *directed adjacency matrix* represents a directed graph, where edges have a direction (arrows). When $A_{ij} = 1$, then an edge exists from vertex i to vertex j .

¹If the edges are weighted, the $\text{matrix}[i][j]$ is equal to the weight, or 0 if the edge does not exist.

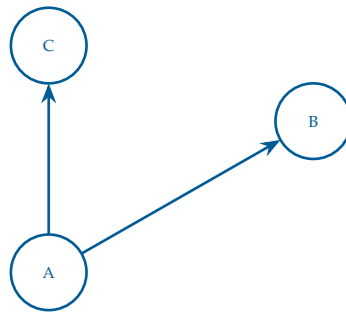


Figure 5.3: A simple directed graph used to make an adjacency matrix.

The resulting adjacency matrix is

$$A = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Notice how, now, $A \neq A^T$, because the edges are directional.

Exercise 6 Adjacency Matrix Exercise

Working Space

The following is an adjacency matrix for a weighted (undirected) graph.

	A	B	C	D	E	F	G
A	0	4	2	0	0	0	0
B	4	0	1	5	0	0	0
C	2	1	0	8	10	0	0
D	0	5	8	0	2	6	0
E	0	0	10	2	0	3	7
F	0	0	0	6	3	0	4
G	0	0	0	0	7	4	0

Draw the corresponding graph with weighted edges.

Answer on Page 47

5.2 Prim's Algorithm and Kruskal's Algorithm for Minimum Spanning Trees

Prim's Algorithm is minimum spanning tree

FIXME <https://github.com/KontinuaFoundation/sequence/issues/315>

5.3 A* (A-star) Search Algorithm

The last shortest path we will discuss in this chapter is the *A-star Algorithm*, more commonly represented as *A* Algorithm*. In essence, it is an extension of Dijkstra's Algorithm in combination with a *priority-based heuristic function*. A heuristic is an introduced estimated cost from any node to the goal.

The key behind this implementation is the following function

$$f(n) = g(n) + h(n)$$

where

- $g(n)$: the cost of the path from the start node (usually S) to the current node n
- $h(n)$: the *heuristic* function that estimates the remaining cost from n to the goal (usually E)
- $f(n) = g(n) + h(n)$: the estimated total cost of a path from S to E passing through node n

5.3.1 The Path Cost $g(n)$

$g(n)$ remains to be the same value that we used in Dijkstra's and other shortest path algorithms.

When moving from a node u to an adjacent node v , the path cost is *updated* as

$$g(v) = g(u) + c(u, v)$$

where $c(u, v)$ is the cost of traversing the edge from u to v . As we discussed in the chapter on Dijkstra's, we have a decision to make upon each visit to a node: Each time we visit a node, we have a decision to make:

$$\text{if } \text{dist}[u] + w(u, v) < \text{dist}[v] \text{ then update } \text{dist}[v]$$

5.3.2 The Heuristic Function $h(n)$

The heuristic $h(n)$ function depends on the problem domain. Its purpose is to estimate the remaining cost from node n to the goal. While this is a difficult task, we can use basic algebra to make an informed guess.

For example, in a map-based pathfinding problem, $h(n)$ may be the straight-line distance, or *Euclidean distance* between node n and the goal. Since the actual path cannot be shorter than the Euclidean distance, this provides a reasonable estimate of the remaining cost. If a nodes location is represented on a 2D plane (such as a map or graph) (x_n, y_n) and the goal location as (x_g, y_g) , the Euclidean Heuristic

$$h(n) = \sqrt{(x_g - x_n)^2 + (y_g - y_n)^2}$$

Alternatively, many 2D games use a similar either *taxicab distance* (typically for 4-way movement),

$$h(n) = |x_g - x_n| + |y_g - y_n|$$

or *Chebyshev distance* (typically for 8-way movement in a game)

$$d = \max(|x_2 - x_1|, |y_2 - y_1|)$$

which takes the maximum of the horizontal and vertical movements.

Sometimes, graphs without direct spatial meaning become more difficult to work with because there is no obvious notion of distance between two nodes.

For general graphs, such as social networks, communication networks, or dependency graphs, the heuristic must be derived from problem-specific knowledge. For example, a social network application might use information such as mutual connections, community membership, or other metadata to estimate how *close* a user is to a target user.

In many cases, however, no suitable heuristic is known. In these situations, the heuristic is often defined as

$$h(n) = 0$$

for all nodes n . Substituting this into the A* evaluation function gives

$$f(n) = g(n)$$

which causes A* to reduce to Dijkstra's algorithm. While this guarantees an optimal path, it loses the efficiency gains that a good heuristic can provide.

There is an important property that the Heuristic Function must satisfy:

$$h(n) \leq \text{true remaining cost}$$

When an admissible heuristic is used (such that $h(n) \leq \text{true remaining cost}$), A^* is guaranteed to find an optimal path if one exists.

5.3.3 The Total Estimated Distance $f(n)$ and Priority Queues

The purpose of the function

$$f(n) = g(n) + h(n)$$

is to assign a priority to each node during the search. Recall from our discussion of priority queues that each element is associated with a priority value, and the element with the highest (or lowest) priority is removed first.

In A^* , the priority of a node is its estimated total distance $f(n)$. Nodes with smaller values of $f(n)$ are considered more promising because they represent paths that appear likely to reach the goal with a lower total cost.

As with Dijkstra's algorithm, A^* maintains a priority queue containing nodes that have been discovered but not yet fully explored. However, instead of prioritizing nodes solely by their known distance from the start node, A^* prioritizes nodes according to their estimated total distance

$$\text{priority}(n) = f(n)$$

At each step, the node with the smallest $f(n)$ value is removed from the priority queue and explored. When a neighboring node is discovered, its path cost $g(n)$ is updated, a new heuristic estimate $h(n)$ is calculated, and a new value of $f(n)$ is computed.

There are some good references about this in your digital resources, as well as a fun Minecraft-based explanation [here](#).

5.4 Injection, Surjection, Bijection, and Isomorphism

Let's talk about Graph Isomorphism. To do this, we must first introduce the terms of *injection*, *surjection*, and *bijection*. These are types of functions in set theory as well as mathematics.

Definition 1 (Injection). *A function $f : A \rightarrow B$ is injective if different inputs always produce different outputs. Importantly, this means no two integers map to the same output.*

Definition 2 (Surjection). *A function $f : A \rightarrow B$ is surjective if every element of B is hit by at*

least one element of A . In essence, all of the elements of B are covered.

Definition 3 (Bijection). A function $f : A \rightarrow B$ is *bijection* if it is both injective and surjective. Ultimately, the function creates no collisions and hits all functions of B .

Exercise 7 **Bijection, Surjection, Injection Exercise**

Determine whether each function is injective, surjective, bijective, or none of these.

Working Space

1. $f : \{1, 2, 3\} \rightarrow \{a, b, c, d\}$ defined by

$$f(1) = a, \quad f(2) = b, \quad f(3) = c.$$

2. $g : \{1, 2, 3, 4\} \rightarrow \{a, b, c\}$ defined by

$$g(1) = a, \quad g(2) = b, \quad g(3) = c, \quad g(4) = a.$$

3. $h : \{1, 2, 3\} \rightarrow \{a, b, c\}$ defined by

$$h(1) = b, \quad h(2) = c, \quad h(3) = a.$$

Answer on Page 47

Now, let's talk about *Isomorphism*. A *graph isomorphism* is a bijection between the vertex sets of two graphs that preserves adjacency. Thus, an isomorphism not only matches vertices one-to-one, but also preserves the structure of the graph.

Definition 4 (Graph Isomorphism). Let $G = (V_G, E_G)$ and $H = (V_H, E_H)$ be graphs. An *isomorphism from G to H* is a bijection

$$f : V_G \rightarrow V_H$$

such that for all vertices $u, v \in V_G$,

$$\{u, v\} \in E_G \iff \{f(u), f(v)\} \in E_H.$$

If such a function exists, we say that G and H are isomorphic and write

$$G \cong H.$$

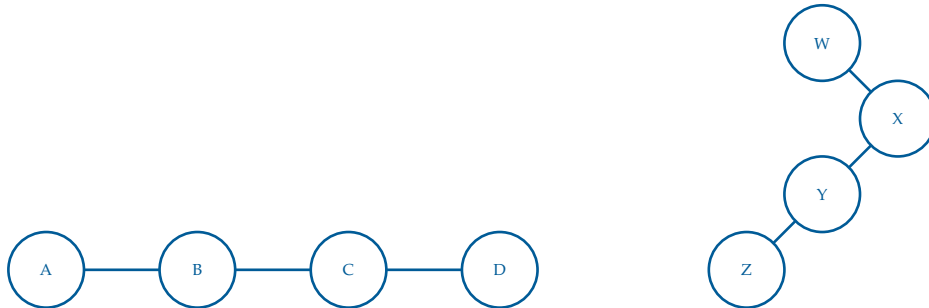


Figure 5.4: Two graphs that are isomorphic. Graph G (left) and Graph H (right).

We are able to say that these graphs (Figure 5.4) are isomorphic as one vertex from G can be paired with one vertex from H, while maintaining the same edge connections.

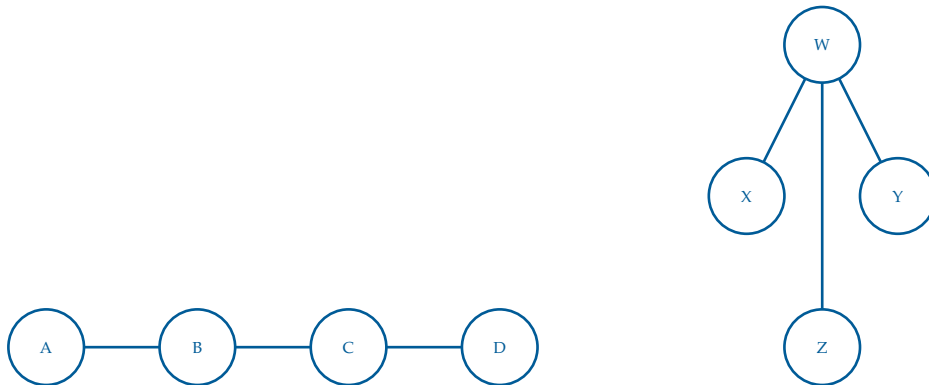


Figure 5.5: Graph X and Y; not isomorphic.

In Figure 5.5, we can see the vertices are not isomorphic without adding or subtracting edges. For example, Graph X does not have a vertex with 3 edges, unlike Graph Y.

If any of these differ, the graphs are not isomorphic:

- Number of vertices
- Number of edges
- Degree sequence (if degree sequences are intact, it does not guarantee isomorphism)
- Number of connected components
- Number of cycles of a given length

- Chromatic number

Example.

A common example that you will not be tested on but is important to know is the following. We may need to introduce a definition first.

Definition 5 (Clique). *A clique is a set of vertices in a graph such that every pair of distinct vertices is connected by an edge.*

Definition 6 (Clique Subgraph). *A k -clique is exactly a subgraph isomorphic to the complete graph K_k . A 3-clique would be a clique of size 3, called K_3 .*

Now, given two graphs G and H , how can we determine G contain a subgraph isomorphic to H , a graph of smaller to equal size to G ?

This may be easy if G and H are pretty small, but the math becomes infinitely harder as the number of vertices and edges go up.

We can simplify this by transforming the inputs of a clique problem to inputs of the subgraph isomorphism problem.

The clique problem gives us a graph M and a threshold k . The output of this is whether M contains a clique of size at least k .

To transform this into an instance of the subgraph isomorphism problem, construct a smaller subgraph $H = K_k$, the complete graph on k vertices, and let $G = M$, the larger graph.

We now ask the subgraph isomorphism question:

Does G contain a subgraph isomorphic to H ?

Since H is the complete graph K_k , a subgraph of G isomorphic to H must contain k vertices where every pair of vertices is connected by an edge. By definition, this is exactly a k -clique.

Therefore,

$$M \text{ contains a } k\text{-clique} \iff G \text{ contains a subgraph isomorphic to } K_k.$$

This shows that every instance of the Clique problem can be transformed into an instance of the Subgraph Isomorphism problem. We call this an *algorithmic reduction*.²

²Algorithmic Reductions are the basis of a set of problems called P, NP, and NP-Complete. See [this video](#)

5.5 Travelling Salesperson Problem and Nearest Neighbour

FIXME <https://github.com/KontinuaFoundation/sequence/issues/312>

5.6 Topological Sort and DAGs

Another common use of graph algorithms is sorting them in a meaningful order. Specifically, we can sort directed graphs that tells us exactly what order vertices need to be “visited” or operated on. We call this a *Topological Sorting* of the graph. A *topological order* is a linear ordering of the vertices in a graph such that for every directed edge $u \rightarrow v$ from vertex u to vertex v , vertex u comes before vertex v in the ordering. The most common usage for this is scheduling jobs, queueing tasks with dependencies, or creating tentative project schedules.

To do this, we are going to use Python’s Graphlib, and its TopologicalSorter module.

In the graphlib module, graphs are defined in the following format:

```
1 {
2   node: {predecessors}
3 }
```

A basic example of this is the following

```
1 {
2   "C": {"A", "B"}
3 }
```

Which says that A and B need to come before C, such that one possible ordering is

$$A \rightarrow B \rightarrow C$$

Now suppose we alter the graph format as follows:

```
1 {
2   "C": {"A", "B"},
3   "A": {"Z"},
4   "B": {"Z"},
5   "Z": {"C"}
6 }
```

for more.

Here,

- C depends on A and B
- A and B depend on Z
- Z depends on C

There is no way to determine a *starting point*. Now we have an ordering that contains a cycle. This implies that C must come before itself, which is impossible in a linear ordering. Therefore no topological ordering can exist.

Definition 7 (Cycle). A cycle is a valid walk $(v_1, v_2, \dots, v_n)$ where the only repeated vertex is $v_n = v_1$, meaning the last vertex equals the first vertex.

So, we cannot do a topological sort on a graph that contains a cycle. This provides us with a theorem:

Topological Sorting and DAGs

A directed graph admits a topological ordering if and only if it is a *Directed Acyclic Graph* (DAG). That is, the directed graph may not contain any cycles (acyclic).

In fact, the python graphlib library will not produce a valid Topological Sort if a cycle exists. Running the following code

```
1 from graphlib import TopologicalSorter
2 graph = {
3     "C": {"A", "B"},
4     "A": {"Z"},
5     "B": {"Z"},
6     "Z": {"C"}
7 }
8 list(TopologicalSorter(graph).static_order())
```

will produce the following error

```
1 graphlib.CycleError: ('nodes are in a cycle', ['C', 'Z', 'A', 'C'])
```

The exception reports one cycle that prevents the ordering from being constructed. In this case, $C \rightarrow Z \rightarrow A \rightarrow C$ forms a directed cycle.

We can also introduce tasks that can be done in parallel. Create the following file `parallel_dag.py`

```
1 from graphlib import TopologicalSorter
2 tasks = {
3     "Compile": set(),
4     "Build Frontend": {"Compile"},
5     "Build Backend": {"Compile"},
6     "Test": {"Build Frontend", "Build Backend"},
7     "Deploy": {"Test"},
8 }from graphlib import TopologicalSorter
9 tasks = {
10     "Compile": set(),
11     "Build Frontend": {"Compile"},
12     "Build Backend": {"Compile"},
13     "Test": {"Build Frontend", "Build Backend"},
14     "Deploy": {"Test"},
15 }
16 from graphlib import TopologicalSorter
17
18 ts = TopologicalSorter(tasks)
19 ts.prepare()
20
21 while ts.is_active():
22     ready = ts.get_ready()
23     print(ready)
24
25     for task in ready:
26         ts.done(task)
27
```

The 5 nodes here are tasks for a basic software project. The ordering is as follows:

Compile $\rightarrow$ Build Frontend and Build Backend $\rightarrow$ Test $\rightarrow$ Deploy

However notice that *Build Frontend* and *Build Backend* are not dependent on each other, so they could be done in *parallel*. The *Test* task cannot begin until both builds have finished. Running this code outputs the following

```
1 ('Compile',)
2 ('Build Frontend', 'Build Backend')
3 ('Test',)
4 ('Deploy',)
```

showing us that our sort outputs which ones can happen at the same time. In software development, many tasks have dependencies but can still be executed in parallel when those dependencies are satisfied. Topological sorting helps determine both the required execution order and which tasks may be performed concurrently (this is similar to running circuits in parallel!).

5.7 Proofs

This section will be a set of proofs or algorithm design problems for graph-related concepts that are important to know. At the basis level, these are just practice problems for graph understand, but can be applied in many real world scenarios.

5.7.1 Example 1: Bipartite

Bipartite Graph

A *Bipartite Graph* is a graph $G = (V, E)$ such that V can be partitioned into two sets ($V = V_1 \cup V_2$) and ($V_1 \cap V_2 = \text{empty}$) such that there are no edges between vertices in the same set. **FIXME** Diagram

Theorem 1. *An undirected graph G is bipartite if and only if it contains no cycles of odd length.*

Exercise 8 Algorithm for Determining Bipartite

Design pseudocode for an algorithm that determines whether or not an undirected graph is bipartite. *Hint: use the theorem.*

Working Space

Hint 2: This function may be useful

FindOdd(G, e)

Require: G , a graph on which DFS has been run; $e = (x, y)$, a back edge

Ensure: **true** if e creates a cycle of odd length, **false** otherwise

```

1: if  $\text{pre}(x) - \text{post}(y) \equiv 0 \pmod{2}$  then
2:   return true
3: else
4:   return false
5: end if
```

Answer on Page 48

While you don't have to go through it on your own, here is the outline of how we can do a proof by contradiction for a graph, specifically proving theorem 1.

Proof. First, suppose G is bipartite. Then its vertices can be partitioned into two sets V_1 and V_2 , with every edge going between V_1 and V_2 . Any cycle in G must alternate between vertices in V_1 and vertices in V_2 . Therefore, after an odd number of edges, the cycle would end in the opposite set from where it started, so it cannot return to its starting vertex. Hence every cycle has even length.

Conversely, suppose G contains no cycles of odd length. Choose any connected component of G , and pick a vertex s . Put each vertex at even distance from s in V_1 , and each vertex at odd distance from s in V_2 . We claim this gives a valid bipartition.

Assume for contradiction that there is an edge (u, v) where u and v are in the same set. Then u and v have distances from s with the same parity. Taking shortest paths from s to u and from s to v , together with the edge (u, v) , creates a cycle of odd length. This contradicts the assumption that G has no odd cycles.

Thus no edge connects two vertices in the same set, so each connected component is bipartite. Applying this construction to every connected component of G , the whole graph G is bipartite.

Therefore, G is bipartite if and only if it contains no cycles of odd length. □

5.7.2 Example 2 and 3. Cycle Detection and Leaves

Exercise 9 Cycle Detection

Design pseudocode for an algorithm which takes as an input an undirected graph, G , and an edge, $e = (u, v)$, and determines whether G has a cycle containing e .

Working Space

Answer on Page 48

Exercise 10 **Leaf Removal**

Proof the following statement: In any connected undirected graph, there is a vertex whose removal leaves the graph connected.

Working Space

Answer on Page 48

Sometimes, we may need to create new graphs as counterexamples to disprove a statement. Consider the following:

Theorem 2. *An graph G is strongly connected if every vertex is reachable from every other vertex.*

Another way to say this is that a select vertex of the graph has a path to every other vertex on the graph. Often, this comes up in finding *subgraphs* of graphs that are strongly connected, called *strongly connected components*.

Exercise 11 **Strongly Connected Graphs**

Create an example of a strongly connected directed graph $G = (V, E)$ such that, for every $v \in V$, removing v from G leaves a directed graph that is not strongly connected.

*Hint: the simplest answer is usually the best.
Start with one vertex.*

Working Space

Answer on Page 49

Answers to Exercises

Answer to Exercise 1 (on page 7)

The path to `data.csv` is

```
./Documents/Projects/project2/data.csv
```

The depth is the number of edges from the root to the node.

In this case, it is the number of subdirectories needed to reach `data.csv`:

- (1) `./` $\rightarrow$ `Documents`
- (2) `Documents` $\rightarrow$ `Projects`
- (3) `Projects` $\rightarrow$ `project2`
- (4) `project2` $\rightarrow$ `data.csv`

So `data.csv` is at a depth of 4.

Answer to Exercise 2 (on page 15)

Algorithm: DFS (Pre-order) run recursively on a tree

```
procedure DFS(root)
  if root = null then
    return
  end if
  visit(root)
  DFS(root.left)
  DFS(root.right)
end procedure
```

▷ V

▷ L

▷ R

This algorithm defines a depth-first search on a binary tree. Because the node is visited before its subtrees, it corresponds specifically to a pre-order traversal.

Answer to Exercise 3 (on page 16)

```
procedure DFS(root)
  if root = null then
    return
  end if
  pre(root) = counter
  counter = counter + 1
  DFS(root.left)
  DFS(root.right)
  post(root) = counter
  counter = counter + 1
end procedure
```

Answer to Exercise 4 (on page 16)

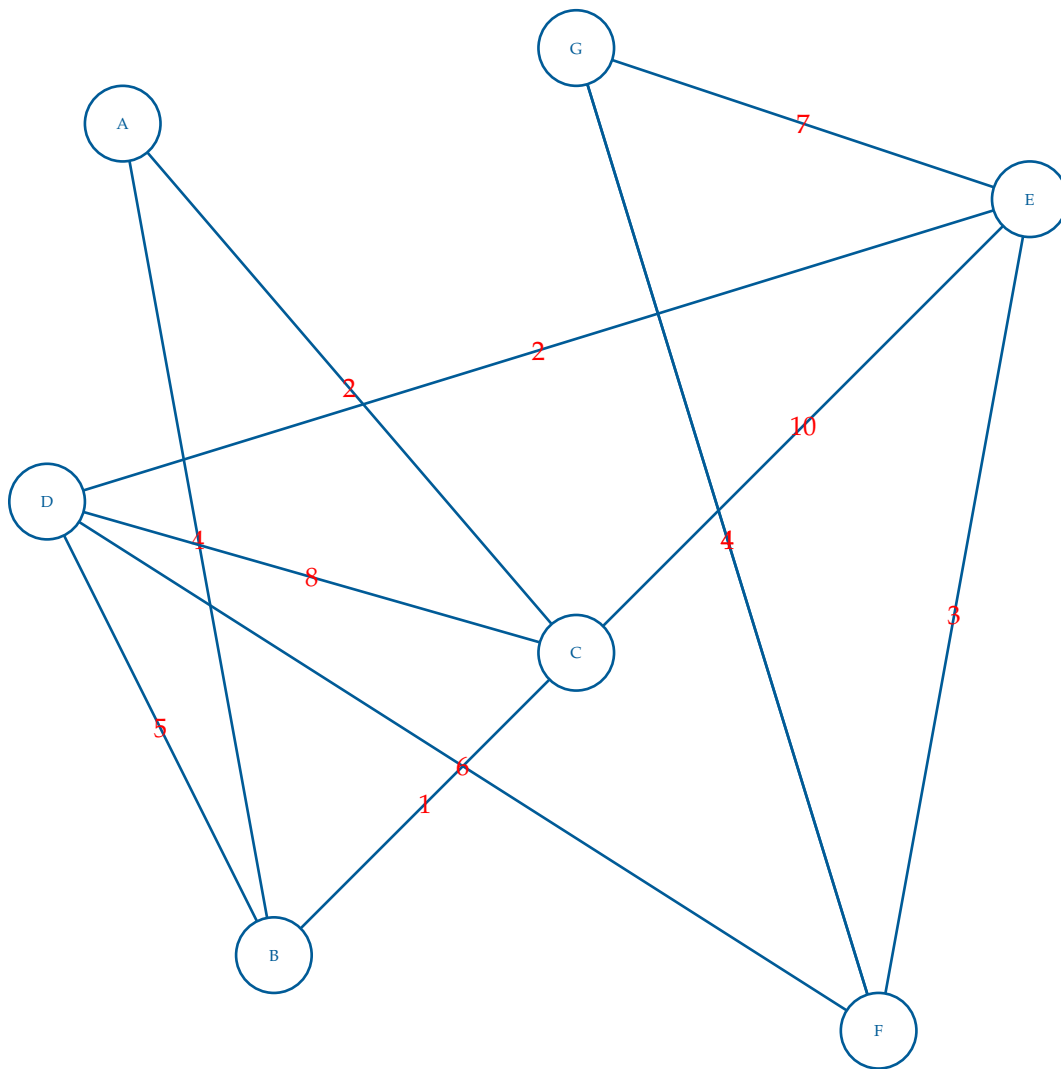
Node	Pre-order Number	Post-order Number
Alan	1	14
Brad	2	7
Dean	3	4
Emily	5	6
Connie	8	13
Faith	9	10
Gordon	11	12

Answer to Exercise 5 (on page 20)

Los Angeles $\rightarrow$ Bakersfield is not chosen because it does not give the best known cost to Bakersfield. The prev pointer for Bakersfield goes to San Diego instead, since $\$21 < \22 .

The tree that is generated is called a *predecessor tree* or *explored tree* of the resulting edges formed by only the previous edges. We can use this to track shortest paths from a given origin node.

Answer to Exercise 6 (on page 31)



Answer to Exercise 7 (on page 35)

1. Injective, but not surjective. No two inputs map to the same output, but d is never reached.
2. Surjective, but not injective. Every element of the codomain is reached, but both 1 and 4 map to a .
3. Bijective. Every element of the codomain is reached exactly once.

Answer to Exercise 8 (on page 41)

Assuming FindOdd exists, we can make the following pseudocode.

IsBipartite(G)

Require: G , a graph on which DFS has been run

Ensure: **true** if G is bipartite, **false** otherwise

```
1: for all back edges  $e$  do
2:   if FINDODD( $G, e$ ) = true then
3:     return false
4:   end if
5: end for
6: return true
```

Answer to Exercise 9 (on page 42)

Remove the edge $e = (u, v)$ from G .

Run BFS or DFS starting from u .

if v is reached **then**

 Return TRUE.

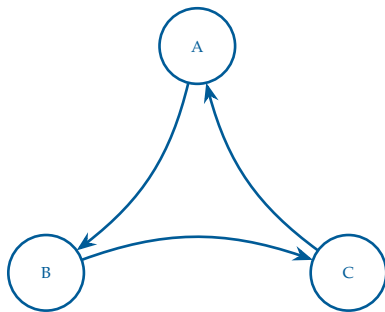
else

 Return FALSE.

end if

Answer to Exercise 10 (on page 43)

Proof. Run DFS on given graph G . Find a leaf, l in the output DFS tree (sometimes called the explore tree). Remove l from the graph. Run DFS on the graph ($G' = G/\{l\}$). If the DFS tree is connected, then we are done. If not, then there is a vertex v in the DFS tree that is not connected to the rest of the graph. This means that v was only connected to l in the original graph. But this contradicts the fact that l was a leaf in the DFS tree, since v would have been explored before l . Since a leaf must exist, removing it from the explore tree does not create a new tree. Therefore, there must be a vertex whose removal leaves the graph connected. $\square$

Answer to Exercise 11 (on page 43)



INDEX

- A* Algorithm, [32](#)
- A-star Algorithm, [32](#)
- adjacency matrix, [30](#)
- algorithmic reduction, [37](#)
- Ancestor, [4](#)

- Bellman-Ford algorithm, [24](#)
- BFS, [9](#)
- bijection, [34](#)
- binary tree, [4](#)
- Bipartite Graph, [41](#)
- breadth-first search, [9](#)

- Chebyshev distance, [33](#)
- cycle, [39](#)
- Cycle Error, [39](#)

- depth, [4](#)
- depth-first search, [9](#)
- Descendant, [4](#)
- DFS, [9](#)
- Dijkstra's Algorithm, [17](#), [18](#)
- Dijkstra, Edsger, [18](#)
- Directed Acyclic Graph, [39](#)
- directed adjacency matrix, [30](#)
- directory, [6](#)

- edges, [3](#)
- Euclidean distance, [33](#)
- explored tree, [46](#)

- FIFO, [10](#)

- folder, [6](#)
- forest, [3](#)

- graph isomorphism, [35](#)

- height, [4](#)
- heuristic, [32](#)

- in-order, [9](#)
- injection, [34](#)
- Isomorphism, [35](#)
- iterative, [14](#)

- leaves, [4](#)
- left child, [4](#)
- level, [4](#)
- level-order, [9](#)

- matrix, [30](#)

- Path, [4](#)
- post-order, [9](#)
- post-order number, [15](#)
- pre-order, [9](#)
- pre-order number, [15](#)
- predecessor tree, [46](#)

- queue, [10](#)

- recursive, [3](#), [14](#)
- recursive traversal, [14](#)
- right child, [4](#)
- root, [3](#)

siblings, [3](#)
strongly connected, [43](#)
strongly connected components, [43](#)
subtree, [3](#)
surjection, [34](#)

taxicab distance, [33](#)
topological order, [38](#)
Topological Sorting, [38](#)
traversals, [9](#)
tree, [3](#)